

Problem pięciu filozofów

Szymon Iwaniuk

Teoria Współbieżności 2025/2026, laboratorium 5
8 listopada 2026

Spis treści

1 Wstęp **3**
1.1 Treść zadania . . . 3
1.2 Opis rozwiązania . . . 3

2 Specyfikacja narzędzi i sprzętu **4**
2.1 Metoda pomiaru . . . 4

3 Algorytmy **5**
3.1 Rozwiązanie trywialne . . . 6
3.2 Rozwiązanie z widelcami podnoszonymi jednocześnie . . . 6
3.3 Rozwiązanie z lokajem . . . 7
3.4 Rozwiązanie asymetryczne (autorskie) . . . 7

4 Implementacja **8**

5 Wyniki **20**
5.1 Blokada w rozwiązaniu trywialnym . . . 20
5.2 Przepustowość . . . 20
5.3 Czas oczekiwania . . . 22
5.4 Sprawiedliwość . . . 22
5.5 Zagłodzenie przez sąsiadów . . . 23

6 Zmiany w konfiguracji problemu **24**
6.1 Dodatkowy (lub brakujący) filozof 24
6.2 Lokaj skonfigurowany dla innej liczby filozofów 25
6.3 Dodatkowy widelec 25
6.4 Inne zmiany konfiguracji 27
6.5 Podsumowanie 28

7 Wnioski **28**

1 Wstęp

1.1 Treść zadania

Celem ćwiczenia jest zapoznanie się z jednym z klasycznych problemów teorii współbieżności – problemem pięciu filozofów (autor E.W. Dijkstra).



Rysunek 1: Problem pięciu filozofów

- Każdy filozof zajmuje się głównie myśleniem.
- Od czasu do czasu potrzebuje zjeść.
- Do jedzenia potrzebne mu są oba widelce po jego prawej i lewej stronie.
- Jedzenie trwa skończoną (ale nieokreśloną z góry) ilość czasu, po czym filozof widelce odkłada i wraca do myślenia.
- Cykl powtarza się od początku.

Ćwiczenia (na podstawie szkieletu programu `Fil5mon.java`):

a) Zaimplementować trywialne rozwiązanie z symetrycznymi filozofami. Zaobserwować problem blokady.

b) Zaimplementować rozwiązanie z widelcami podnoszonymi jednocześnie. Jaki problem może tutaj wystąpić?

c) Zaimplementować rozwiązanie z lokajem.

d) Wykonać pomiary dla każdego rozwiązania i wywnioskować, co ma wpływ na wydajność każdego rozwiązania.

Dodatkowe zadanie: zaproponować autorskie rozwiązanie, inne niż powyższe.

1.2 Opis rozwiązania

Porównujemy cztery implementacje stołu, wszystkie oparte na monitorach Javy (`synchronized`, `wait()`, `notify()`):

- **a) trywialne** – każdy filozof podnosi najpierw lewy, potem prawy widelec,
- **b) oba widelce naraz** – jeden monitor stołu, filozof czeka, aż oba jego widelce będą wolne, i zabiera je w jednym kroku,
- **c) lokaj** – jak a), ale do stołu jednocześnie dopuszczonych jest co najwyżej $n - 1$ filozofów,
- **asymetryczne (autorskie)** – każdy filozof podnosi najpierw widelec o mniejszym numerze, więc ostatni filozof podnosi widelce w odwrotnej kolejności niż pozostali.

Program działa dla dowolnej liczby filozofów n, a nie tylko dla pięciu, dzięki czemu można zbadać, jak rozwiązania skalują się z wielkością stołu.

Razem ze sprawozdaniem zawarty jest kod w języku `Java` zawierający wszystkie implementacje, program wykonujący pomiary oraz skrypt służący do generowania zawartych w dokumencie wykresów.

2 Specyfikacja narzędzi i sprzętu

Program został napisany w języku `Java 27` i uruchomiony na maszynie wirtualnej `OpenJDK`, na komputerze z procesorem `Apple M4` (architektura `ARM64`, 10 rdzeni: 4 wydajnościowe i 6 energooszczędnych), `16 GB` zunifikowanej pamięci `LPDDR5` i systemem operacyjnym `macOS 26`. Do wygenerowania wykresów użyty został `Python` z biblioteką `matplotlib`.

2.1 Metoda pomiaru

Filozof w pętli myśli, staje się głodny, zdobywa oba widelce, je i odkłada widelce. Czasy myślenia i jedzenia są losowane z rozkładu jednostajnego na przedziale $[0, 2t)$, gdzie t to średni czas danego scenariusza. Opóźnienie realizujemy przez `LockSupport.parkNanos`, więc wątek nie zajmuje procesora. Badamy cztery scenariusze (średni czas myślenia / jedzenia):

- **0/0** – bez myślenia i jedzenia, mierzymy wyłącznie koszt synchronizacji,
- **0/1** – myślenie 0 ms, jedzenie 1 ms, filozofowie są cały czas głodni (największa rywalizacja),
- **1/1** – myślenie 1 ms, jedzenie 1 ms,
- **10/1** – myślenie 10 ms, jedzenie 1 ms (mała rywalizacja).

Każdy przebieg trwa 1 s, a wszystkie wątki startują jednocześnie po zwolnieniu zatrzasku `CountDownLatch`. Każdy filozof liczy swoje posiłki oraz czas oczekiwania na widelce, czyli czas od zgłoszenia głodu do rozpoczęcia jedzenia (`System.nanoTime()`). Na tej podstawie wyznaczamy przepustowość (liczba posiłków na sekundę), średni i maksymalny czas oczekiwania oraz indeks Jaina liczby posiłków

$$J(x_1, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2},$$

gdzie x_i to liczba posiłków i-tego filozofa. $J = 1$, gdy wszyscy zjedli tyle samo razy, a $J = \frac{1}{n}$, gdy jadł tylko jeden filozof. Przed pomiarami wykonujemy kilka przebiegów rozgrzewających kompilator JIT, a każdą konfigurację uruchamiamy trzy razy i bierzemy medianę. Parametry: $n \in \{5, 10, 20, 50, 100\}$.

Rozwiązanie trywialne badamy osobno: przebieg kończy się w chwili wykrycia blokady (albo po 1 s), a każdą konfigurację $n \in \{5, 10, 20, 50\}$ uruchamiamy 20 razy. Blokadę wykrywamy dokładnie: stół liczy filozofów, którzy trzymają lewy widelec i czekają na prawy. Gdy jest ich n, wszystkie widelce są zajęte i żaden filozof nie może kontynuować.

Ostatni eksperyment sprawdza zagłodzenie: przy $n = 5$ filozofowie F1 i F3 („żarłoki") w ogóle nie myślą i jedzą średnio 2 ms, a pozostali myślą 10 ms i jedzą 1 ms. Przebieg trwa 3 s.

W każdym przebiegu filozof po zdobyciu widelców zapisuje się w tablicy właścicieli jako właściciel każdego z nich (metodą `compareAndSet`) i sprawdza, czy żaden z tych widelców nie należy już do innego jedzącego filozofa. Takie zdarzenie przerwałoby pomiary wyjątkiem – we wszystkich przebiegach nie wystąpiło. Przebieg, w którym filozof nie zakończył pracy w ciągu 5 s po zatrzymaniu (niewykryta blokada), również przerwałby pomiary.

Eksperymenty ze zmienioną konfiguracją problemu opisujemy w rozdziale 6.

3 Algorytmy

Filozof i korzysta z widelca i (lewego) i widelca $(i + 1) \bmod n$ (prawego). Wszystkie rozwiązania różnią się tylko procedurami ACQUIRE i RELEASE, pętla filozofa jest wspólna.

Pętla filozofa

```
1: Procedura FILOZOF(i)
2:     Powtarzaj
3:         myśl
4:         t_0 ← NOW()
5:         ACQUIRE(i)
6:         zapisz czas oczekiwania NOW() − t_0
7:         jedz
8:         RELEASE(i)
```

Widelec w rozwiązaniach a), c) i asymetrycznym jest monitorem działającym jak semafor binarny.

Widelec

```
1: Procedura TAKE(w) ▷ synchronized
2:     Dopóki w.zajęty powtarzaj
3:         WAIT()
4:     w.zajęty ← true
5: Procedura PUT(w) ▷ synchronized
6:     w.zajęty ← false, NOTIFY()
```

3.1 Rozwiązanie trywialne

Rozwiązanie trywialne

```
1: Procedura ACQUIRE(i)
2:     TAKE(widelec[i])
3:     TAKE(widelec[(i + 1) mod n])
4: Procedura RELEASE(i)
5:     PUT(widelec[(i + 1) mod n]),  PUT(widelec[i])
```

Jeżeli wszyscy filozofowie jednocześnie wykonają wiersz 2, każdy trzyma swój lewy widelec, a jego prawy widelec jest lewym widelcem sąsiada. Spełnione są wszystkie cztery warunki Coffmana: wzajemne wykluczanie, trzymanie zasobu podczas oczekiwania, brak wywłaszczania oraz cykl oczekiwania $F_0 \to F_1 \to \cdots \to F_{n-1} \to F_0$. Dochodzi do blokady (zakleszczenia).

3.2 Rozwiązanie z widelcami podnoszonymi jednocześnie

Widelce podnoszone jednocześnie

```
1: Procedura ACQUIRE(i) ▷ synchronized na całym stole
2:     Dopóki zajęty[i] ∨ zajęty[(i + 1) mod n] powtarzaj
3:         WAIT()
4:     zajęty[i] ← true,  zajęty[(i + 1) mod n] ← true
5: Procedura RELEASE(i) ▷ synchronized na całym stole
6:     zajęty[i] ← false,  zajęty[(i + 1) mod n] ← false
7:     NOTIFYALL() ▷ obaj sąsiedzi mogą czekać
```

Filozof nigdy nie trzyma jednego widelca, czekając na drugi, więc warunek „trzymaj i czekaj” nie jest spełniony i blokada jest niemożliwa. Pojawia się za to **zagłodzenie**: filozof i może jeść tylko wtedy, gdy jednocześnie nie je żaden z jego sąsiadów. Jeżeli sąsiedzi jedzą na zmianę tak, że zawsze je co najmniej jeden z nich, filozof i czeka w nieskończoność, mimo że cały system działa. Dodatkowo cały stół jest jednym monitorem, a NOTIFYALL budzi wszystkich czekających, chociaż jeść mogą co najwyżej dwaj sąsiedzi zwalniającego.

3.3 Rozwiązanie z lokajem

Rozwiązanie z lokajem

```
1: Procedura ACQUIRE(i)
2:     synchronized(lokaj): dopóki przy stole = n − 1 WAIT();  przy stole ← przy stole + 1
3:     TAKE(widelec[i])
4:     TAKE(widelec[(i + 1) mod n])
5: Procedura RELEASE(i)
6:     PUT(widelec[(i + 1) mod n]),  PUT(widelec[i])
7:     synchronized(lokaj): przy stole ← przy stole − 1,  NOTIFY()
```

Przy stole jest co najwyżej $n-1$ filozofów, a widelców jest n. Z zasady szufladkowej co najmniej jeden filozof przy stole ma dostęp do obu swoich widelców, więc cykl oczekiwania nie może się zamknąć. Lokaj jest jednak wspólnym monitorem, przez który przechodzi każdy posiłek.

3.4 Rozwiązanie asymetryczne (autorskie)

Rozwiązanie asymetryczne (hierarchia zasobów)

```
1: Procedura ACQUIRE(i)
2:     a ← min(i, (i + 1) mod n),  b ← max(i, (i + 1) mod n)
3:     TAKE(widelec[a]),  TAKE(widelec[b])
4: Procedura RELEASE(i)
5:     PUT(widelec[b]),  PUT(widelec[a])
```

Widelce są ponumerowane, a każdy filozof podnosi je w kolejności rosnących numerów. Dla filozofów $0, \ldots, n-2$ oznacza to lewy, potem prawy widelec, a filozof $n-1$ (widelce $n-1$ i 0) podnosi najpierw prawy. W cyklu oczekiwania każdy wątek czekałby na widelec o większym numerze niż ten, który trzyma, co jest niemożliwe w zamkniętym cyklu. Rozwiązanie nie wymaga żadnego wspólnego monitora – każdy widelec synchronizuje tylko dwóch sąsiadów.

4 Implementacja

Poniżej znajduje się pełny kod programu. Każda klasa jest w osobnym pliku w katalogu `src`.
Klasa **Fork** to widelec ze szkieletu programu (`Widelec`) zrealizowany jako monitor.

```
/** Fork as a Java monitor, like Widelec from the skeleton: take() waits until the fork is free. */
class Fork {
    private boolean taken = false;

    synchronized void take() throws InterruptedException {
        while (taken) wait();
        taken = true;
    }

    synchronized void put() {
        taken = false;
        notify();
    }
}
```

Kod 1: Klasa **Fork** (`src/Fork.java`)

Klasa **Table** jest wspólną bazą wszystkich rozwiązań. Metoda `create` wybiera implementację po nazwie.

```
/**
 * Common base of all solutions. Philosopher i uses fork i (left) and fork (i+1) mod n (right);
 * acquire(i) returns when philosopher i holds both forks, release(i) puts them back.
 */
abstract class Table {
    final int n;
    private int[][] pairs;

    Table(int n) {
        this.n = n;
    }

    /** Number of forks on the table, n unless a configuration adds one. */
    int forkCount() {
        return n;
    }

    /** Forks held by philosopher i while eating (used by the mutual exclusion check). */
    int[] forksOf(int i) {
        if (pairs == null) {
            int[][] p = new int[n][];
            for (int j = 0; j < n; j++) p[j] = new int[]{left(j), right(j)};
            pairs = p;
        }
        return pairs[i];
    }

    int left(int i) {
        return i;
    }
```

```
31
32      int right(int i) {
33          return (i + 1) % n;
34      }
35
36      abstract void acquire(int i) throws InterruptedException;
37
38      abstract void release(int i);
39
40      /** True when the table is certainly in a deadlock (only the naive solution can be). */
41      boolean deadlocked() {
42          return false;
43      }
44
45      static Fork[] forks(int n) {
46          Fork[] forks = new Fork[n];
47          for (int i = 0; i < n; i++) forks[i] = new Fork();
48          return forks;
49      }
50
51      static Table create(String impl, int n) {
52          return switch (impl) {
53              case "naive" -> new NaiveTable(n);
54              case "both" -> new BothTable(n);
55              case "waiter" -> new WaiterTable(n);
56              case "asym" -> new AsymmetricTable(n);
57              case "spare" -> new SpareForkTable(n);
58              case "ring" -> new ExtraForkTable(n);
59              default -> throw new IllegalArgumentException(impl);
60          };
61      }
62  }
```

Kod 2: Klasa **Table** (`src/Table.java`)

Klasa **NaiveTable** to rozwiązanie trywialne a). Licznik `holdingOne` służy wyłącznie do wykrywania blokady.

```
1   import java.util.concurrent.atomic.AtomicInteger;
2
3   /** a) Symmetric philosophers: left fork first, then right fork. Can deadlock. */
4   class NaiveTable extends Table {
5       private final Fork[] forks;
6       // philosophers holding their left fork and not yet the right one
7       private final AtomicInteger holdingOne = new AtomicInteger();
8
9       NaiveTable(int n) {
10          super(n);
11          forks = forks(n);
12      }
13
14      @Override
15      void acquire(int i) throws InterruptedException {
16          forks[left(i)].take();
```

```
17          holdingOne.incrementAndGet();
18          forks[right(i)].take();
19          holdingOne.decrementAndGet();
20      }
21
22      @Override
23      void release(int i) {
24          forks[right(i)].put();
25          forks[left(i)].put();
26      }
27
28      /** All n philosophers hold one fork each and wait for the other: nobody can ever continue. */
29      @Override
30      boolean deadlocked() {
31          return holdingOne.get() == n;
32      }
33  }
```

Kod 3: Klasa **NaiveTable** (`src/NaiveTable.java`)

Klasa **BothTable** to rozwiązanie b) z widelcami podnoszonymi jednocześnie.

```
1   /**
2    * b) Both forks taken at once: one monitor for the whole table, a philosopher waits
3    * until both forks are free and takes them in one step. No deadlock, but
4    * a philosopher whose neighbours eat in turns can starve.
5    */
6   class BothTable extends Table {
7       private final boolean[] taken;
8
9       BothTable(int n) {
10          super(n);
11          taken = new boolean[n];
12      }
13
14      @Override
15      synchronized void acquire(int i) throws InterruptedException {
16          while (taken[left(i)] || taken[right(i)]) wait();
17          taken[left(i)] = true;
18          taken[right(i)] = true;
19      }
20
21      @Override
22      synchronized void release(int i) {
23          taken[left(i)] = false;
24          taken[right(i)] = false;
25          notifyAll(); // both neighbours may be waiting
26      }
27  }
```

Kod 4: Klasa **BothTable** (`src/BothTable.java`)

Klasy **Waiter** i **WaiterTable** to rozwiązanie c) z lokajem. Drugi konstruktor pozwala ustawić inny limit, co wykorzystujemy w rozdziale 6.

```java
/** Waiter (lokaj) lets at most limit philosophers try to take forks at the same time. */
class Waiter {
    private final int limit;
    private int seated = 0;

    Waiter(int limit) {
        this.limit = limit;
    }

    synchronized void enter() throws InterruptedException {
        while (seated == limit) wait();
        seated++;
    }

    synchronized void leave() {
        seated--;
        notify(); // exactly one place became free
    }
}
```

Kod 5: Klasa **Waiter** (`src/Waiter.java`)

```java
import java.util.concurrent.atomic.AtomicInteger;

/**
 * c) Naive fork order guarded by a waiter that admits at most n - 1 philosophers.
 * A different limit models a waiter configured for another number of philosophers.
 */
class WaiterTable extends Table {
    private final Fork[] forks;
    private final Waiter waiter;
    private final AtomicInteger holdingOne = new AtomicInteger();

    WaiterTable(int n) {
        this(n, n - 1);
    }

    WaiterTable(int n, int limit) {
        super(n);
        forks = forks(n);
        waiter = new Waiter(limit);
    }

    @Override
    void acquire(int i) throws InterruptedException {
        waiter.enter();
        forks[left(i)].take();
        holdingOne.incrementAndGet();
        forks[right(i)].take();
        holdingOne.decrementAndGet();
    }

    @Override
```

```
32      void release(int i) {
33          forks[right(i)].put();
34          forks[left(i)].put();
35          waiter.leave();
36      }
37
38      /** Possible only with limit >= n: everybody holds the left fork. */
39      @Override
40      boolean deadlocked() {
41          return holdingOne.get() == n;
42      }
43  }
```

Kod 6: Klasa **WaiterTable** (`src/WaiterTable.java`)

Klasa **AsymmetricTable** to rozwiązanie autorskie z hierarchią widelców.

```
1   /**
2    * Own solution: resource ordering. Every philosopher takes the fork with the lower number
3    * first, so the last one (forks n-1 and 0) takes the right fork first. No central monitor.
4    */
5   class AsymmetricTable extends Table {
6       private final Fork[] forks;
7
8       AsymmetricTable(int n) {
9           super(n);
10          forks = forks(n);
11      }
12
13      @Override
14      void acquire(int i) throws InterruptedException {
15          forks[Math.min(left(i), right(i))].take();
16          forks[Math.max(left(i), right(i))].take();
17      }
18
19      @Override
20      void release(int i) {
21          forks[Math.max(left(i), right(i))].put();
22          forks[Math.min(left(i), right(i))].put();
23      }
24  }
```

Kod 7: Klasa **AsymmetricTable** (`src/AsymmetricTable.java`)

Klasy **SpareForkTable** i **ExtraForkTable** to konfiguracje z dodatkowym widelcem z rozdziału 6.

```
1   /**
2    * Extra fork in the middle of the table: naive order (left fork, then right fork), but a philosopher
3    * whose right fork is taken may use the spare fork instead. n philosophers hold at most n forks
4    * while waiting, there are n + 1, so somebody always gets two and the cycle cannot close.
5    * One monitor for the whole table, because a philosopher waits for one of two forks.
6    */
7   class SpareForkTable extends Table {
8       private final boolean[] taken; // ring forks 0..n-1, spare fork n
9       private final int[][] held;    // forks currently held by every philosopher
```

```
10
11      SpareForkTable(int n) {
12          super(n);
13          taken = new boolean[n + 1];
14          held = new int[n][2];
15      }
16
17      @Override
18      int forkCount() {
19          return n + 1;
20      }
21
22      @Override
23      int[] forksOf(int i) {
24          return held[i];
25      }
26
27      @Override
28      synchronized void acquire(int i) throws InterruptedException {
29          while (taken[left(i)]) wait();
30          taken[left(i)] = true;
31          while (taken[right(i)] && taken[n]) wait();
32          int second = taken[right(i)] ? n : right(i);
33          taken[second] = true;
34          held[i][0] = left(i);
35          held[i][1] = second;
36      }
37
38      @Override
39      synchronized void release(int i) {
40          taken[held[i][1]] = false;
41          taken[held[i][0]] = false;
42          notifyAll();
43      }
44  }
```

Kod 8: Klasa **SpareForkTable** (`src/SpareForkTable.java`)

```
1   /**
2    * Extra fork in the ring: n philosophers and n + 1 forks, philosopher i uses forks i and i + 1,
3    * so philosophers n - 1 and 0 do not share a fork. Naive order (left, then right) but the waiting
4    * graph is a path, not a cycle, so there is no deadlock.
5    */
6   class ExtraForkTable extends Table {
7       private final Fork[] forks;
8
9       ExtraForkTable(int n) {
10          super(n);
11          forks = forks(n + 1);
12      }
13
14      @Override
15      int forkCount() {
```

```
16          return n + 1;
17      }
18
19      @Override
20      int right(int i) {
21          return i + 1;
22      }
23
24      @Override
25      void acquire(int i) throws InterruptedException {
26          forks[left(i)].take();
27          forks[right(i)].take();
28      }
29
30      @Override
31      void release(int i) {
32          forks[right(i)].put();
33          forks[left(i)].put();
34      }
35  }
```

Kod 9: Klasa **ExtraForkTable** (`src/ExtraForkTable.java`)

Klasa **Philosopher** to wątek filozofa ze szkieletu (`Filozof`) rozszerzony o pomiary i sprawdzenie, czy żaden widelec nie jest używany przez dwóch filozofów jednocześnie.

```
1   import java.util.SplittableRandom;
2   import java.util.concurrent.CountDownLatch;
3   import java.util.concurrent.atomic.AtomicIntegerArray;
4   import java.util.concurrent.locks.LockSupport;
5
6   /**
7    * Thinks and eats for random times (uniform on [0, 2 * mean)) until stop is set,
8    * recording the number of meals and the time spent waiting for forks.
9    */
10  class Philosopher extends Thread {
11      private final int id;
12      private final Table table;
13      private final long thinkNs, eatNs;
14      private final SplittableRandom rnd;
15      private final AtomicIntegerArray owner; // owner + 1 of every fork, 0 when free; checks mutual
        ↪ exclusion
16      private final CountDownLatch start;
17      private volatile boolean stop = false;
18      long meals = 0, waitSumNs = 0, waitMaxNs = 0;
19
20      Philosopher(int id, Table table, long thinkNs, long eatNs, long seed,
21                  AtomicIntegerArray owner, CountDownLatch start) {
22          this.id = id;
23          this.table = table;
24          this.thinkNs = thinkNs;
25          this.eatNs = eatNs;
26          this.rnd = new SplittableRandom(seed);
27          this.owner = owner;
```

```
28          this.start = start;
29      }
30
31      void finish() {
32          stop = true;
33      }
34
35      private void pause(long meanNs) {
36          if (meanNs == 0) return;
37          long end = System.nanoTime() + rnd.nextLong(2 * meanNs);
38          long left;
39          while ((left = end - System.nanoTime()) > 0) LockSupport.parkNanos(left);
40      }
41
42      @Override
43      public void run() {
44          try {
45              start.await();
46              while (!stop) {
47                  pause(thinkNs);                                    // thinking
48                  long t0 = System.nanoTime();
49                  table.acquire(id);                                 // hungry
50                  long wait = System.nanoTime() - t0;
51                  int[] forks = table.forksOf(id);
52                  for (int f : forks)
53                      if (!owner.compareAndSet(f, 0, id + 1))
54                          throw new IllegalStateException("fork " + f + " used by " + id + " and " + (
    ↪ owner.get(f) - 1));
55                  meals++;
56                  waitSumNs += wait;
57                  waitMaxNs = Math.max(waitMaxNs, wait);
58                  pause(eatNs);                                      // eating
59                  for (int f : forks) owner.set(f, 0);
60                  table.release(id);
61              }
62          } catch (InterruptedException e) {
63              // stopped while waiting, only happens after a deadlock
64          }
65      }
66  }
```

Kod 10: Klasa **Philosopher** (`src/Philosopher.java`)

Klasa **Main** wykonuje wszystkie eksperymenty i zapisuje wyniki do plików CSV, z których skrypt w Pythonie rysuje wykresy.

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.PrintWriter;
4  import java.util.Arrays;
5  import java.util.concurrent.CountDownLatch;
6  import java.util.concurrent.atomic.AtomicIntegerArray;
7  import java.util.concurrent.atomic.AtomicReference;
8  import java.util.concurrent.locks.LockSupport;
```

```
9
10 /**
11  * Three experiments, results in csv/:
12  * runs.csv       - deadlock-free solutions, throughput, waiting time and fairness vs n and scenario,
13  * deadlock.csv   - naive solution, time until deadlock,
14  * starvation.csv - n = 5 with two greedy philosophers, meals of every philosopher,
15  * config.csv     - changed configurations: other number of philosophers, stale waiter limit, extra fork
    ↪ .
16  */
17 public class Main {
18     static final String[] IMPLS = {"both", "waiter", "asym"};
19     static final int[] SIZES = {5, 10, 20, 50, 100};
20     static final int[] DEADLOCK_SIZES = {5, 10, 20, 50};
21     static final long MS = 1_000_000L;
22     /** name, mean thinking time, mean eating time [ns] */
23     record Scenario(String name, long thinkNs, long eatNs) {}
24     static final Scenario[] SCENARIOS = {
25             new Scenario("0/0", 0, 0),
26             new Scenario("0/1", 0, MS),
27             new Scenario("1/1", MS, MS),
28             new Scenario("10/1", 10 * MS, MS),
29     };
30     static final long RUN_NS = 1000 * MS;
31     static final int REPS = 3;
32     static final int DEADLOCK_REPS = 20;
33     static final long STARVE_NS = 3000 * MS;
34     static final int CONFIG_DEADLOCK_REPS = 20;
35
36     record Result(long timeNs, boolean deadlock, long[] meals, double[] perMeanUs, double[] perMaxUs) {
37         long totalMeals() {
38             return Arrays.stream(meals).sum();
39         }
40
41         double meanWaitUs() {
42             double sum = 0;
43             for (int i = 0; i < meals.length; i++) sum += perMeanUs[i] * meals[i];
44             return totalMeals() == 0 ? 0 : sum / totalMeals();
45         }
46
47         double maxWaitUs() {
48             return Arrays.stream(perMaxUs).max().orElse(0);
49         }
50
51         /** Jain's index of meals: 1 when everybody ate the same number of times, 1/n when one ate. */
52         double jain() {
53             double sum = 0, sumSq = 0;
54             for (long m : meals) {
55                 sum += m;
56                 sumSq += (double) m * m;
57             }
58             return sumSq == 0 ? 0 : sum * sum / (meals.length * sumSq);
59         }
60     }
```

```
61
62      static Result run(String impl, long[] think, long[] eat, long limitNs, long seed) throws
        ↪ InterruptedException {
63          return run(Table.create(impl, think.length), think, eat, limitNs, seed);
64      }
65
66      static Result run(Table table, long[] think, long[] eat, long limitNs, long seed) throws
        ↪ InterruptedException {
67          int n = think.length;
68          String impl = table.getClass().getSimpleName();
69          AtomicIntegerArray owner = new AtomicIntegerArray(table.forkCount());
70          AtomicReference<Throwable> error = new AtomicReference<>();
71          CountDownLatch start = new CountDownLatch(1);
72          Philosopher[] ph = new Philosopher[n];
73          for (int i = 0; i < n; i++) {
74              ph[i] = new Philosopher(i, table, think[i], eat[i], seed * 1_000 + i, owner, start);
75              ph[i].setUncaughtExceptionHandler((t, e) -> error.compareAndSet(null, e));
76              ph[i].start();
77          }
78          long t0 = System.nanoTime();
79          start.countDown();
80          boolean deadlock = false;
81          while (System.nanoTime() - t0 < limitNs) {
82              if (table.deadlocked()) {
83                  deadlock = true;
84                  break;
85              }
86              LockSupport.parkNanos(100_000);
87          }
88          long elapsed = System.nanoTime() - t0;
89          for (Philosopher p : ph) p.finish();
90          if (deadlock) for (Philosopher p : ph) p.interrupt();
91          for (Philosopher p : ph) {
92              p.join(5_000);
93              if (p.isAlive()) throw new IllegalStateException(impl + ": philosopher did not finish (
        ↪ undetected deadlock)");
94          }
95          if (error.get() != null) throw new IllegalStateException(impl, error.get());
96
97          long[] meals = new long[n];
98          double[] mean = new double[n], max = new double[n];
99          for (int i = 0; i < n; i++) {
100             meals[i] = ph[i].meals;
101             mean[i] = meals[i] == 0 ? 0 : ph[i].waitSumNs / 1e3 / meals[i];
102             max[i] = ph[i].waitMaxNs / 1e3;
103         }
104         return new Result(elapsed, deadlock, meals, mean, max);
105     }
106
107     static void config(PrintWriter w, String experiment, String variant, Scenario s, Result res, int rep
        ↪ ) {
108         double sec = res.timeNs() / 1e9;
```

```
109         w.printf("%s,%s,%s,%d,%d,%d,%.3f,%.1f,%.3f,%.3f,%.4f,%s%n", experiment, variant, s.name(), res.
    ↪ meals().length, rep,
110                 res.deadlock() ? 1 : 0, res.timeNs() / 1e6, res.totalMeals() / sec, res.meanWaitUs(),
    ↪ res.maxWaitUs(),
111                 res.jain(), Arrays.stream(res.meals()).mapToObj(Long::toString).reduce((a, b) -> a + ";"
    ↪  + b).orElse(""));
112         System.out.printf("%-12s %-7s %-5s n=%d rep %2d: %s %8.0f meals/s, J=%.3f, meals %s%n",
    ↪ experiment, variant, s.name(),
113                 res.meals().length, rep, res.deadlock() ? "DEADLOCK" : "ok      ", res.totalMeals() /
    ↪ sec, res.jain(),
114                 Arrays.toString(res.meals()));
115     }
116
117     static long[] same(int n, long v) {
118         long[] a = new long[n];
119         Arrays.fill(a, v);
120         return a;
121     }
122
123     public static void main(String[] args) throws IOException, InterruptedException {
124         String out = args.length > 0 ? args[0] : "csv";
125
126         // JIT warm-up, results discarded
127         for (String impl : IMPLS)
128             for (int i = 0; i < 3; i++) run(impl, same(10, 0), same(10, 0), 300 * MS, i);
129
130         try (PrintWriter w = new PrintWriter(new FileWriter(out + "/runs.csv"))) {
131             w.println("impl,scenario,n,rep,time_ms,meals,throughput,mean_wait_us,max_wait_us,jain");
132             for (Scenario s : SCENARIOS)
133                 for (int n : SIZES)
134                     for (String impl : IMPLS)
135                         for (int r = 0; r < REPS; r++) {
136                             Result res = run(impl, same(n, s.thinkNs()), same(n, s.eatNs()), RUN_NS, r);
137                             if (res.deadlock()) throw new IllegalStateException(impl + " deadlocked");
138                             double sec = res.timeNs() / 1e9;
139                             w.printf("%s,%s,%d,%d,%.3f,%d,%.1f,%.3f,%.3f,%.4f%n", impl, s.name(), n, r,
140                                     res.timeNs() / 1e6, res.totalMeals(), res.totalMeals() / sec,
141                                     res.meanWaitUs(), res.maxWaitUs(), res.jain());
142                             System.out.printf("%-5s n=%-3d %-6s rep %d: %10.0f meals/s, wait %9.1f us, J
    ↪ =%.3f%n",
143                                     s.name(), n, impl, r, res.totalMeals() / sec, res.meanWaitUs(), res.
    ↪ jain());
144                         }
145         }
146
147         try (PrintWriter w = new PrintWriter(new FileWriter(out + "/deadlock.csv"))) {
148             w.println("scenario,n,rep,deadlock,time_ms,meals");
149             for (Scenario s : SCENARIOS)
150                 for (int n : DEADLOCK_SIZES)
151                     for (int r = 0; r < DEADLOCK_REPS; r++) {
152                         Result res = run("naive", same(n, s.thinkNs()), same(n, s.eatNs()), RUN_NS, r);
153                         w.printf("%s,%d,%d,%d,%.3f,%d%n", s.name(), n, r, res.deadlock() ? 1 : 0,
154                                 res.timeNs() / 1e6, res.totalMeals());
```

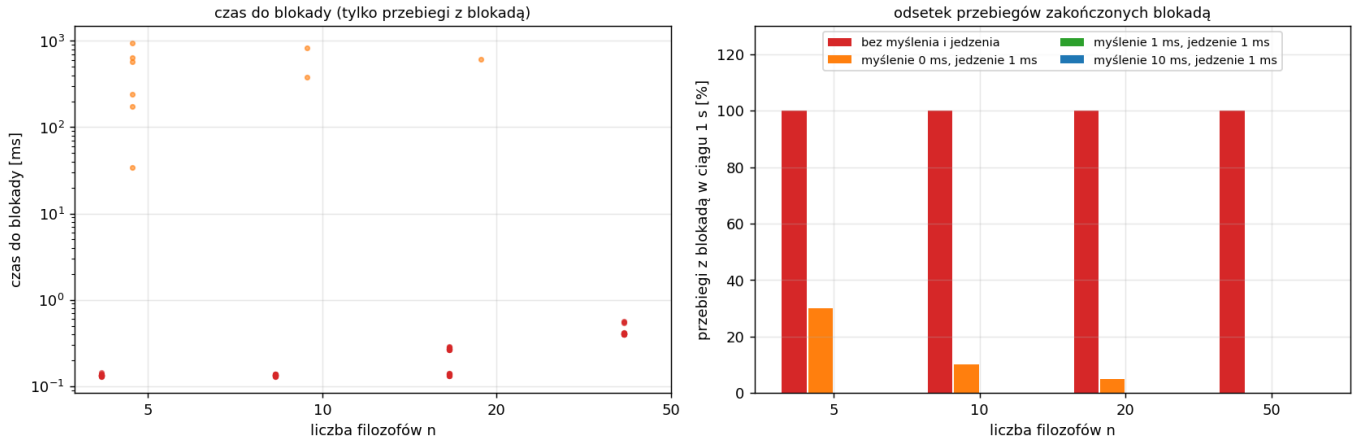
```
155                         System.out.printf("%-5s n=%-3d naive  rep %2d: %s after %8.2f ms, %d meals%n", s
    ↪ .name(), n, r,
156                                 res.deadlock() ? "DEADLOCK" : "no deadlock", res.timeNs() / 1e6, res.
    ↪ totalMeals());
157                     }
158             }
159
160             // philosophers 1 and 3 never think and eat 2 ms, the others think 10 ms and eat 1 ms
161             long[] think = {10 * MS, 0, 10 * MS, 0, 10 * MS};
162             long[] eat = {MS, 2 * MS, MS, 2 * MS, MS};
163             try (PrintWriter w = new PrintWriter(new FileWriter(out + "/starvation.csv"))) {
164                 w.println("impl,rep,philosopher,meals,mean_wait_us,max_wait_us");
165                 for (String impl : IMPLS)
166                     for (int r = 0; r < REPS; r++) {
167                         Result res = run(impl, think, eat, STARVE_NS, r);
168                         for (int i = 0; i < 5; i++)
169                             w.printf("%s,%d,%d,%d,%.3f,%.3f%n", impl, r, i, res.meals()[i], res.perMeanUs()[
    ↪ i], res.perMaxUs()[i]);
170                         System.out.printf("starvation %-6s rep %d: meals %s%n", impl, r, Arrays.toString(res
    ↪ .meals()));
171                     }
172             }
173             try (PrintWriter w = new PrintWriter(new FileWriter(out + "/config.csv"))) {
174                 w.println("experiment,variant,scenario,n,rep,deadlock,time_ms,throughput,mean_wait_us,
    ↪ max_wait_us,jain,meals");
175                 Scenario[] contended = {SCENARIOS[1], SCENARIOS[2]};
176                 // one philosopher fewer or more than five
177                 for (Scenario s : contended)
178                     for (int n : new int[]{4, 5, 6})
179                         for (String impl : new String[]{"naive", "both", "waiter", "asym"})
180                             for (int r = 0; r < (impl.equals("naive") ? CONFIG_DEADLOCK_REPS : REPS); r++)
181                                 config(w, "philosophers", impl, s, run(impl, same(n, s.thinkNs()), same(n, s
    ↪ .eatNs()), RUN_NS, r), r);
182                 // waiter still configured for five philosophers (limit 4)
183                 for (Scenario s : contended)
184                     for (int n : new int[]{4, 6})
185                         for (int r = 0; r < CONFIG_DEADLOCK_REPS; r++)
186                             config(w, "waiter_limit", "limit4", s,
187                                     run(new WaiterTable(n, 4), same(n, s.thinkNs()), same(n, s.eatNs()),
    ↪ RUN_NS, r), r);
188                 // extra fork: none, in the middle of the table, in the ring
189                 for (Scenario s : new Scenario[]{SCENARIOS[0], SCENARIOS[1], SCENARIOS[2]})
190                     for (String impl : new String[]{"naive", "spare", "ring"})
191                         for (int r = 0; r < CONFIG_DEADLOCK_REPS; r++)
192                             config(w, "extra_fork", impl, s, run(impl, same(5, s.thinkNs()), same(5, s.eatNs
    ↪ ()), RUN_NS, r), r);
193             }
194         }
195 }
```

Kod 11: Klasa **Main** (`src/Main.java`)

5 Wyniki

5.1 Blokada w rozwiązaniu trywialnym

Rysunek 2 pokazuje czas do wystąpienia blokady oraz odsetek przebiegów, w których blokada wystąpiła w ciągu 1 s.



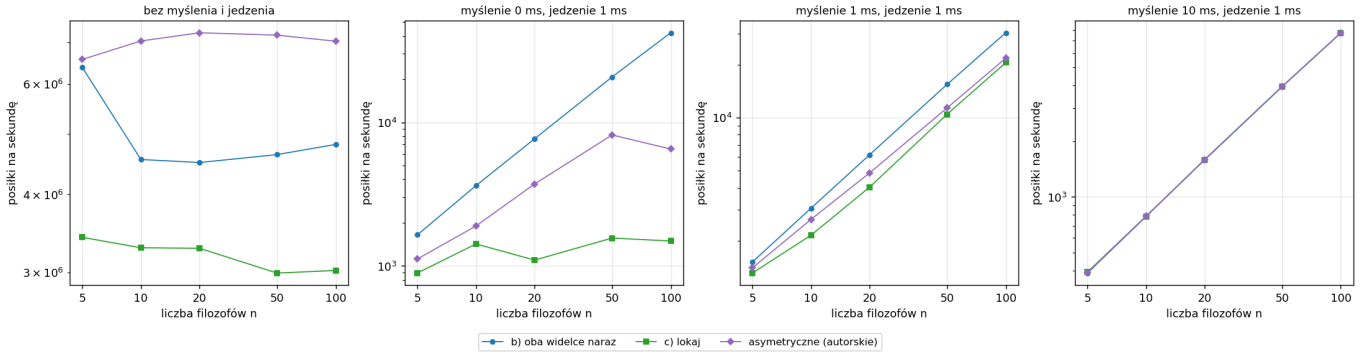
Rysunek 2: Blokada w rozwiązaniu trywialnym (20 przebiegów na konfigurację)

Bez myślenia i jedzenia blokada wystąpiła w każdym przebiegu, a mediana czasu do blokady wynosi 0,13 ms dla $n = 5$ i 0,41 ms dla $n = 50$ (filozofowie zdążyli zjeść łącznie odpowiednio 133 i 1706 razy). Przy ciągle głodnych filozofach i jedzeniu 1 ms blokada wystąpiła w 30% przebiegów dla $n = 5$ (mediana 411 ms), w 10% dla $n = 10$, 5% dla $n = 20$, a dla $n = 50$ ani razu. Gdy filozofowie myślą 1 ms lub 10 ms, w tym eksperymencie blokada nie wystąpiła w ciągu sekundy ani razu. Nie oznacza to jednak, że jest niemożliwa: w eksperymencie z rozdziału 6 przy myśleniu 1 ms wystąpiła w 5% przebiegów dla $n = 4$ i w 10% dla $n = 6$. Wyniki z poprzedniego uruchomienia całych pomiarów różniły się wyraźnie (np. 95% zamiast 30% dla scenariusza 0/1 i $n = 5$), więc prawdopodobieństwo blokady w krótkim przebiegu silnie zależy od przypadkowego przeplotu wątków.

Blokada wymaga, żeby wszyscy filozofowie jednocześnie trzymali lewy widelec. Im krótsze myślenie i im mniej filozofów, tym większe szanse na takie ułożenie, dlatego rozwiązanie trywialne wydaje się działać poprawnie przy małej rywalizacji. Nie jest jednak poprawne – blokada jest tylko kwestią czasu.

5.2 Przepustowość

Rysunek 3 pokazuje liczbę posiłków na sekundę dla rozwiązań wolnych od blokady.



Rysunek 3: Przepustowość w zależności od liczby filozofów

Tabela 1 zestawia mediany dla $n = 5$ i $n = 100$.

Scenariusz	**Rozwiązanie**	$n = 5$			$n = 100$		
		posiłki/s	czekanie [µs]	J	posiłki/s	czekanie [µs]	J
0/0	b) oba naraz	6 383 726	0,6	1,00	4 808 533	15,4	0,98
	c) lokaj	3 415 848	1,2	1,00	3 023 800	24,3	0,99
	asymetryczne	6 573 349	0,6	0,51	7 029 457	14,1	0,02
0/1	b) oba naraz	1645	1825,2	0,78	42 067	1213,9	0,50
	c) lokaj	892	4404,4	0,23	1491	70 173,4	0,05
	asymetryczne	1118	3291,9	0,35	6515	15 217,8	0,18
1/1	b) oba naraz	1512	935,8	1,00	30 339	975,1	1,00
	c) lokaj	1309	1440,5	1,00	20 539	2552,1	1,00
	asymetryczne	1401	1163,6	0,99	21 768	2286,7	0,99
10/1	b) oba naraz	388	230,0	1,00	7691	150,5	1,00
	c) lokaj	393	214,9	1,00	7701	150,5	1,00
	asymetryczne	390	239,2	1,00	7687	163,0	1,00

Tabela 1: Przepustowość, średni czas oczekiwania na widelce i indeks Jaina J

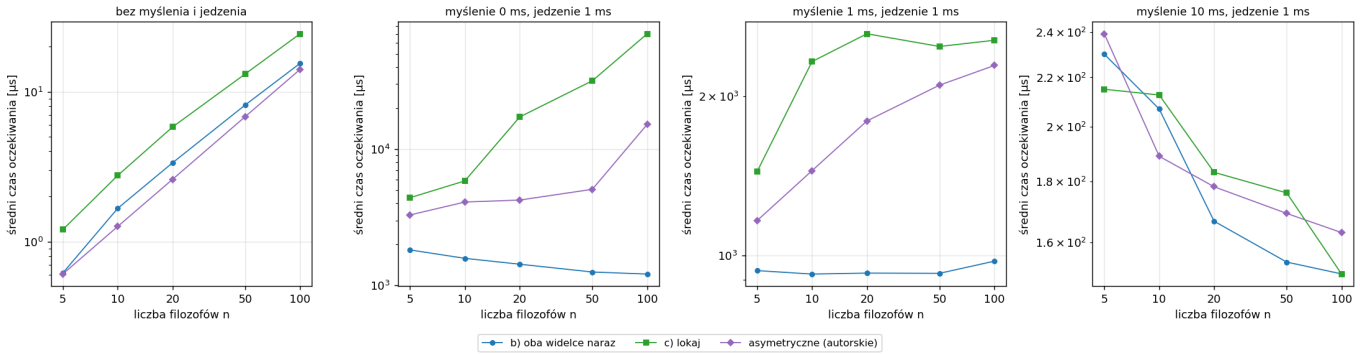
Bez myślenia i jedzenia (0/0) najszybsze jest rozwiązanie asymetryczne: 6,6–7,3 mln posiłków na sekundę, bo każdy widelec synchronizuje tylko dwóch sąsiadów i nie ma wspólnego monitora. Rozwiązanie b) jest wolniejsze (4,5–6,4 mln), a dla $n \geq 10$ wyraźnie traci, bo każdy posiłek przechodzi przez jeden monitor stołu, a `notifyAll()` budzi wszystkich czekających. Lokaj jest najwolniejszy (3,0–3,4 mln) – każdy posiłek to dwa wejścia do wspólnego monitora lokaja i cztery do monitorów widelców.

Przy jedzeniu 1 ms koszt synchronizacji jest pomijalny, a przepustowość zależy od tego, ilu filozofów je jednocześnie. W scenariuszu 10/1 filozofowie rzadko konkurują, wszystkie rozwiązania osiągają ten sam wynik, rosnący liniowo z n (nieco poniżej $\frac{n}{11\,\text{ms}}$ ze względu na narzut `parkNanos`). W scenariuszu 1/1 najlepsze jest rozwiązanie b), bo filozof nie zajmuje widelca, z którego nie może skorzystać. W pozostałych rozwiązaniach filozof trzymający jeden widelec blokuje sąsiada, mimo że sam jeszcze nie je, i powstają łańcuchy oczekujących filozofów.

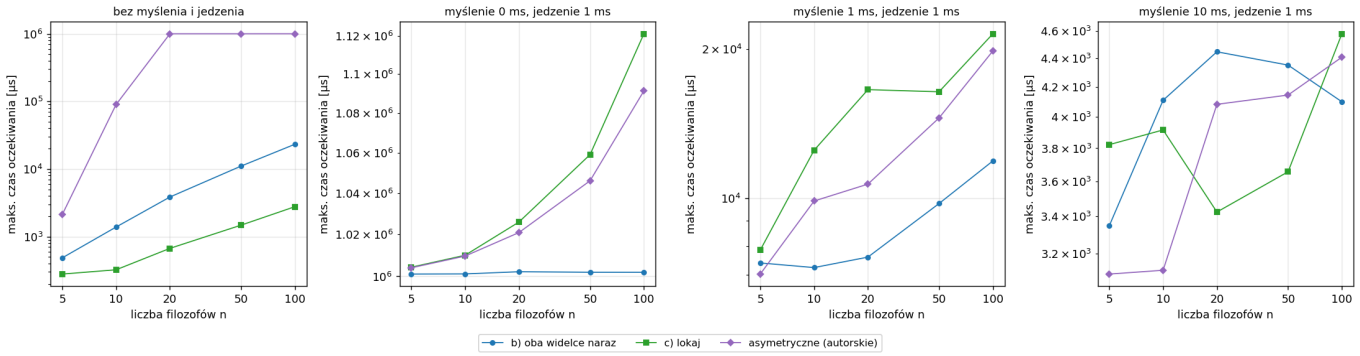
Największe różnice są w scenariuszu 0/1 (ciągle głodni filozofowie). Rozwiązanie b) osiąga dla $n = 100$ 42 tys. posiłków na sekundę, czyli blisko maksimum $\frac{n}{2}$ jedzących naraz (50 tys.). Rozwiązanie asymetryczne osiąga 6,5 tys., a lokaj tylko 1,5 tys. posiłków na sekundę – mniej niż 2 filozofów je jednocześnie. Filozof, który odłożył widelce, natychmiast próbuje podnieść je ponownie, więc niemal każdy widelec jest zajęty przez kogoś, kto czeka na drugi, i tworzą się długie łańcuchy oczekiwania. Przy lokaju dodatkowo kolejność wpuszczania do stołu nie ma związku z dostępnymi widelcami.

5.3 Czas oczekiwania

Rysunki 4 i 5 pokazują średni i maksymalny czas oczekiwania na widelce.



Rysunek 4: Średni czas oczekiwania na widelce

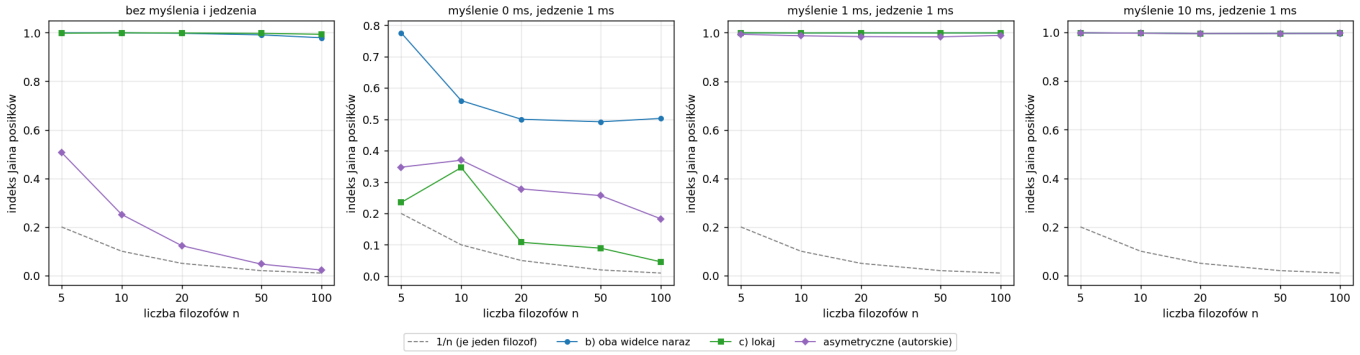


Rysunek 5: Maksymalny czas oczekiwania na widelce

Średnie czasy odpowiadają przepustowości: przy małej rywalizacji (10/1) filozof czeka średnio 150–240 µs, a w scenariuszu 0/1 dla $n = 100$ od 1,2 ms (b) do 70 ms (lokaj). Maksymalny czas oczekiwania w scenariuszu 0/1 dla wszystkich rozwiązań wynosi około 1 s, czyli cały przebieg: niektórzy filozofowie doczekali się widelców dopiero wtedy, gdy pozostali kończyli pracę. Bez myślenia i jedzenia to samo dotyczy rozwiązania asymetrycznego dla $n = 100$.

5.4 Sprawiedliwość

Rysunek 6 pokazuje indeks Jaina liczby posiłków. Przerywana linia $\frac{1}{n}$ odpowiada sytuacji, w której je tylko jeden filozof.



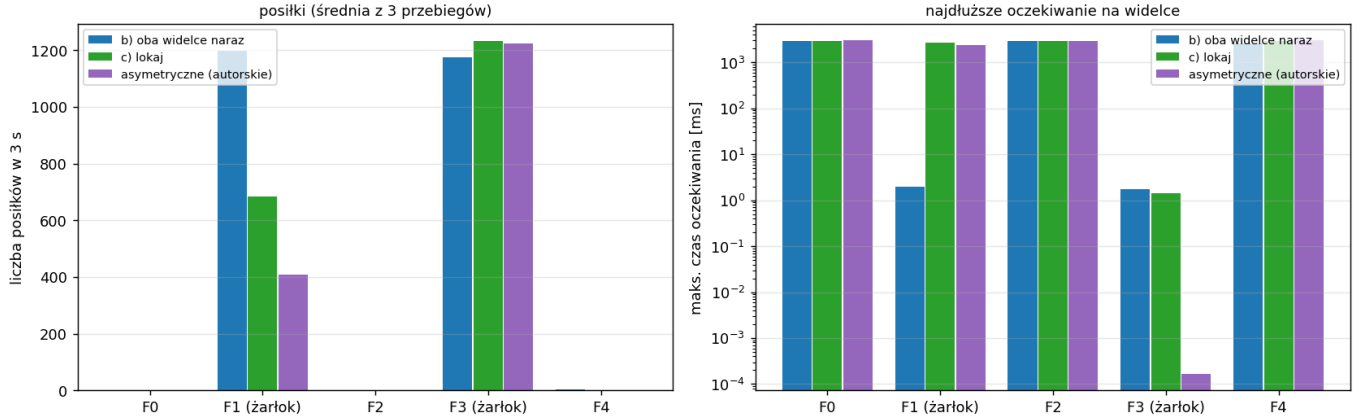
Rysunek 6: Indeks Jaina liczby posiłków

Gdy filozofowie myślą (1/1 i 10/1), wszystkie rozwiązania są sprawiedliwe ($J \geq 0{,}99$). Gdy nie myślą (0/1), wszystkie głodzą filozofów. W rozwiązaniu b) dla $n \geq 10$ $J \approx 0{,}5$: je co drugi filozof, a jego sąsiedzi prawie wcale. W rozwiązaniu z lokajem i asymetrycznym J spada dla $n = 100$ do 0,05 i 0,18, czyli blisko $\frac{1}{n}$. Bez myślenia i jedzenia (0/0) b) i lokaj są sprawiedliwe, a rozwiązanie asymetryczne nie (dla $n = 100$ $J = 0{,}02$).

Przyczyna jest ta sama co w laboratorium 4: monitor Javy nie gwarantuje kolejności wejścia. Filozof, który odłożył widelec (NOTIFY), od razu wywołuje TAKE ponownie i zwykle zdobywa monitor widelca, zanim obudzony sąsiad zdąży się wykonać. Przerwa na myślenie daje sąsiadowi czas na zabranie widelca, dlatego w scenariuszach 1/1 i 10/1 problem znika.

5.5 Zagłodzenie przez sąsiadów

Rysunek 7 pokazuje wynik eksperymentu z dwoma żarłocznymi filozofami F1 i F3.



Rysunek 7: Liczba posiłków i najdłuższe oczekiwanie, F1 i F3 nie myślą, $n = 5$, 3 s

W rozwiązaniu b) żarłoki zjadły średnio 1198 i 1174 razy, a F0, F2 i F4 tylko 2, 1 i 4 razy w ciągu 3 s. Najdłuższe oczekiwanie F2 wyniosło 3,00 s, czyli praktycznie cały przebieg. To dokładnie scenariusz zagłodzenia z rozdziału 3.2: F1 i F3 nie są sąsiadami, więc mogą jeść jednocześnie i na zmianę, a F2 potrzebuje chwili, w której nie je żaden z nich. W rozwiązaniu z lokajem i asymetrycznym F0, F2 i F4 również jedzą tylko kilka razy, a F1 zjada wyraźnie mniej niż F3 (683 i 409 razy wobec ponad 1200): F2 trzyma widelec

2, czekając na widelec 3 zajęty przez F3, więc blokuje też F1. Żadne z badanych rozwiązań nie chroni przed zagłodzeniem, gdy niektórzy filozofowie nie dają innym szansy.

6 Zmiany w konfiguracji problemu

Rozwiązania z rozdziału 3 są poprawne przy założeniach klasycznego problemu: n filozofów przy okrągłym stole, n widelców, każdy filozof potrzebuje dokładnie dwóch sąsiednich widelców, wszyscy działają zgodnie z protokołem i nikt nie odchodzi od stołu. W tym rozdziale sprawdzamy, co się dzieje, gdy któreś z tych założeń się zmieni: pojawi się dodatkowy filozof albo dodatkowy widelec, lokaj zostanie skonfigurowany dla innej liczby filozofów itd. Interesuje nas, czy zmiana zwiększa ryzyko blokady, zagłodzenia oraz jak wpływa na wydajność.

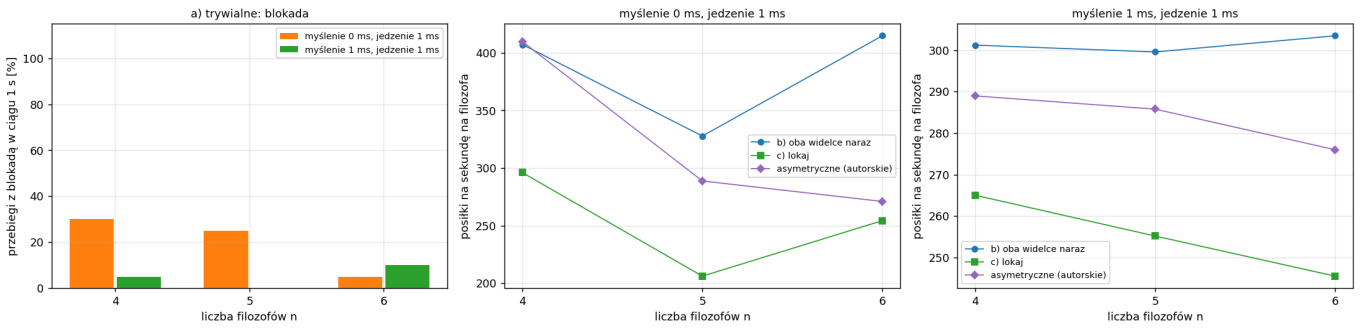
Pomiary wykonujemy tak jak w rozdziale 2.1, w scenariuszach 0/1 i 1/1 (a dla dodatkowego widelca także 0/0). Rozwiązania, w których może dojść do blokady, uruchamiamy 20 razy na konfigurację, pozostałe 3 razy.

6.1 Dodatkowy (lub brakujący) filozof

Zmiana liczby filozofów z 5 na 4 lub 6 oznacza też zmianę liczby widelców, bo każdy nowy filozof przynosi swój widelec. Teoretycznie:

- **a) trywialne** – cykl oczekiwania istnieje dla każdego $n \geq 2$, więc blokada pozostaje możliwa. Prawdopodobieństwo blokady maleje z n, bo wszyscy filozofowie muszą jednocześnie trzymać lewy widelec, ale nigdy nie spada do zera.
- **b) oba widelce naraz** – dalej bez blokady. Dla parzystego n stół można podzielić na filozofów parzystych i nieparzystych. Jeżeli jedna grupa je na zmianę z małymi przerwami, druga może być zagłodzona w całości. Dla nieparzystego n taki podział nie istnieje.
- **c) lokaj** – poprawny dla każdego n, ale tylko wtedy, gdy limit lokaja to $n - 1$ (rozdział 6.2).
- **asymetryczne** – poprawne dla każdego n, bo porządek widelców jest globalny. Nowy widelec dostaje kolejny numer i nie trzeba zmieniać pozostałych filozofów.

Liczba filozofów jedzących jednocześnie wynosi co najwyżej $\lfloor n/2 \rfloor$: dla $n = 4$ i $n = 5$ są to dwaj filozofowie, a dla $n = 6$ trzej. Dodanie piątego filozofa do czterech nie zwiększa więc przepustowości, a dodanie szóstego do pięciu ją zwiększa.



Rysunek 8: Rozwiązanie trywialne i przepustowość na filozofa dla $n \in \{4, 5, 6\}$

Pomiary to potwierdzają. W rozwiązaniu trywialnym w scenariuszu 0/1 blokada wystąpiła w 30%, 25% i 5% przebiegów dla $n = 4, 5, 6$, a w scenariuszu 1/1 w 5%, 0% i 10% przebiegów. Większa liczba filozofów zmniejsza więc ryzyko blokady, ale go nie usuwa. W rozwiązaniu b) w scenariuszu 0/1 przepustowość wynosi 1629, 1639 i 2489 posiłków na sekundę dla $n = 4, 5, 6$, czyli szósty filozof zwiększa ją o 52%, a piąty nie zmienia jej wcale. Na wykresie widać to jako spadek liczby posiłków na filozofa dla $n = 5$ ($\lfloor n/2 \rfloor / n$ wynosi 0,5, 0,4 i 0,5). W scenariuszu 1/1 rywalizacja jest mała i liczba posiłków na filozofa prawie nie zależy od n.

Parzystość ma też wpływ na zagłodzenie. W rozwiązaniu b) dla $n = 6$ w scenariuszu 0/1 filozofowie F1, F3 i F5 zjedli 724, 826 i 751 razy, a F2 i F4 tylko raz – dokładnie zagłodzenie jednej z dwóch grup. Dla $n = 4$ jest podobnie (269, 542, 283 i 538 posiłków), a dla $n = 5$ podział jest mniej wyraźny (od 135 do 508 posiłków).

6.2 Lokaj skonfigurowany dla innej liczby filozofów

Lokaj ma limit wpisany na stałe dla pięciu filozofów (4). Jeżeli liczba filozofów się zmieni, a limit nie, możliwe są dwa przypadki:

- **mniej filozofów ($n = 4$, limit 4)** – limit nie jest mniejszy od n, więc lokaj wpuszcza wszystkich i rozwiązanie staje się trywialne, z możliwą blokadą. W pomiarach blokada wystąpiła w 5% przebiegów zarówno w scenariuszu 0/1, jak i 1/1, a bez myślenia i jedzenia po ułamku milisekundy.
- **więcej filozofów ($n = 6$, limit 4)** – rozwiązanie pozostaje poprawne, bo przy stole jest mniej niż n filozofów, ale lokaj niepotrzebnie zatrzymuje dwóch filozofów zamiast jednego. W scenariuszu 0/1 przepustowość spadła z 1526 do 1153 posiłków na sekundę (o 24%), a indeks Jaina z 0,37 do 0,34. W scenariuszu 1/1 różnica była w granicach błędu (1552 wobec 1473).

Zbyt duży limit jest więc błędem poprawności, a zbyt mały kosztuje wydajność. Limit należy wyznaczać z aktualnej liczby filozofów, a przy filozofach dosiadających się w trakcie działania zmieniać go razem z nią pod tym samym monitorem.

6.3 Dodatkowy widelec

Dodatkowy widelec można położyć w dwóch miejscach, a skutki są różne.

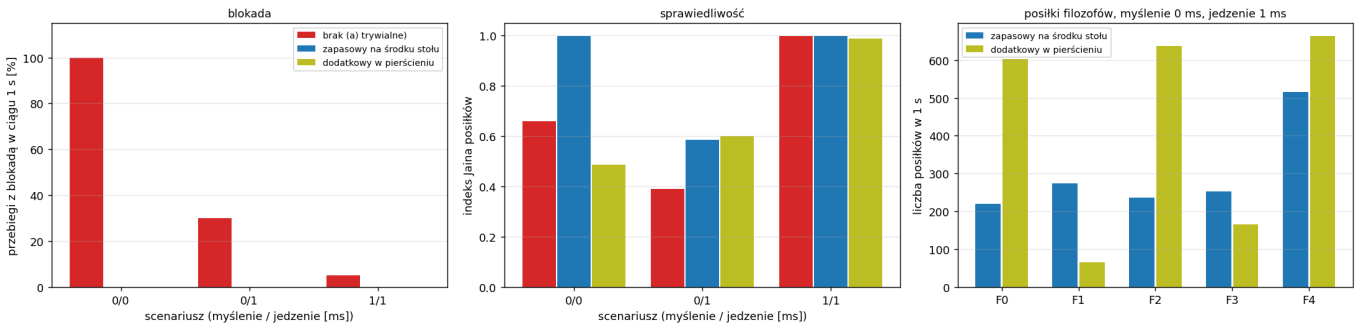
Zapasowy widelec na środku stołu. Filozof podnosi lewy widelec, a jeżeli prawy jest zajęty, może użyć zapasowego (klasa **SpareForkTable**). Ponieważ filozof może czekać na jeden z dwóch widelców, cały stół jest jednym monitorem.

Rozwiązanie trywialne z zapasowym widelcem

```
1:  Procedura ACQUIRE(i) ▷ synchronized na całym stole
2:      Dopóki zajęty[i] powtarzaj
3:          WAIT()
4:      zajęty[i] ← true
5:      Dopóki zajęty[(i + 1) mod n] ∧ zajęty[zapasowy] powtarzaj
6:          WAIT()
7:      d ← jeżeli wolny (i + 1) mod n to (i + 1) mod n inaczej zapasowy
8:      zajęty[d] ← true,  zapamiętaj d
9:  Procedura RELEASE(i) ▷ synchronized na całym stole
10:     zajęty[d] ← false,  zajęty[i] ← false,  NOTIFYALL()
```

Filozofów jest n, a widelców $n+1$, więc z zasady szufladkowej któryś filozof zawsze zdobędzie dwa widelce i blokada jest niemożliwa – to ten sam argument co dla lokaja, tylko zamiast odebrać miejsce przy stole, dodajemy zasób. Zapasowy widelec zwiększa też liczbę jedzących naraz: dla $n = 5$ mogą jeść trzej filozofowie (np. F0, F2 i F4 z zapasowym). Ceną jest wspólny monitor i NOTIFYALL, a zapasowy widelec jest zasobem, o który rywalizują wszyscy.

Dodatkowy widelec w pierścieniu. Widelców jest $n + 1$, a filozof i używa widelców i i $i + 1$ (klasa **ExtraForkTable**). Filozofowie F($n - 1$) i F0 nie mają już wspólnego widelca, więc stół zamienia się w łańcuch. Graf oczekiwania nie może utworzyć cyklu i nawet rozwiązanie trywialne nie zakleszcza się. Filozofowie nie są jednak już równoprawni: F0 i F($n - 1$) mają jednego sąsiada, z którym rywalizują, a pozostali dwóch. Dla $n = 5$ filozofowie F0, F2 i F4 mogą jeść jednocześnie i trwale zagłodzić F1 i F3.



Rysunek 9: Dodatkowy widelec, $n = 5$: blokada, indeks Jaina i liczba posiłków w scenariuszu 0/1

W pomiarach żadna z konfiguracji z dodatkowym widelcem nie zakleszczyła się, podczas gdy rozwiązanie trywialne zablokowało się w 100%, 30% i 5% przebiegów w scenariuszach 0/0, 0/1 i 1/1. Zapasowy widelec dał bez myślenia i jedzenia 6,5 mln posiłków na sekundę przy $J = 1{,}00$, a w scenariuszu 1/1 1657 posiłków na sekundę, więcej niż najlepsze rozwiązanie z rozdziału 5 dla $n = 5$ (b), 1512) – dzięki trzem jedzącym zamiast dwóch. Widelec w pierścieniu dał w scenariuszu 0/1 najwyższą przepustowość (2479 posiłków na sekundę), ale kosztem zagłodzenia: F0, F2 i F4 zjedli 603, 637 i 665 razy, a F1 i F3 tylko 66 i 166 razy. Bez myślenia i jedzenia liczba posiłków rośnie wzdłuż łańcucha od 0,18 mln (F0) do 4,0 mln (F4, który ma własny widelec 5), a $J = 0{,}49$.

6.4 Inne zmiany konfiguracji

Pozostałe zmiany omawiamy bez pomiarów.

- **Brakujący widelec** (np. zgubiony lub uszkodzony) – dwaj filozofowie, którzy go potrzebują, nigdy nie zjedzą, czyli są zagłodzeni przez samą konfigurację. W rozwiązaniach a), c) i asymetrycznym jest gorzej: filozof, który podniósł swój drugi widelec i czeka na brakujący, trzyma go w nieskończoność, więc głoduje też jego sąsiad, a dalej cały łańcuch. W rozwiązaniu b) filozof nie trzyma widelców podczas czekania, więc problem pozostaje lokalny.
- **Filozof, który nie odkłada widelców** (awaria wątku w trakcie jedzenia) – skutek jak wyżej. Monitory Javy nie mają limitu czasu oczekiwania, więc bez `wait(timeout)` lub `tryLock` system nie może z tego wybrnąć.
- **Filozof potrzebujący więcej niż dwóch widelców** – warunek blokady dotyczy dowolnego podzbioru filozofów, którzy trzymają część swoich widelców. Rozwiązanie asymetryczne (podnoszenie w rosnącej kolejności) dalej działa, rozwiązanie b) także, ale limit lokaja $n-1$ nie wystarcza i musi zostać przeliczony. Rośnie rywalizacja i maleje liczba jedzących naraz.
- **Filozof z innym tempem** (żarłok, który nie myśli) – zbadane w rozdziale 5.5: żadne rozwiązanie nie chroni sąsiadów takiego filozofa przed zagłodzeniem.
- **Filozof ignorujący protokół** (np. w rozwiązaniu asymetrycznym podnoszący widelce w innej kolejności albo omijający lokaja) – przywraca cykl oczekiwania i możliwość blokady. Poprawność rozwiązań a), c) i asymetrycznego zależy od tego, czy przestrzegają go wszyscy.
- **Filozofowie dosiadający się i odchodzący w trakcie działania** – zmiana sąsiadów lub widelców filozofa, który właśnie trzyma widelec, prowadzi do błędów wykluczania. Zmianę trzeba wykonywać, gdy dotknięci filozofowie nie jedzą. W rozwiązaniu b) zmianę można wykonać pod monitorem stołu, w rozwiązaniu z lokajem limit musi zmieniać się razem z liczbą filozofów.

6.5 Podsumowanie

Tabela 2 zestawia skutki badanych zmian.

Zmiana	**Blokada**	**Zagłodzenie**	**Wydajność**
dodatkowy filozof	a) dalej możliwa, mniej prawdopodobna; b), c), asym. nie	b): przy parzystym n może głodować co drugi filozof	rośnie, gdy n staje się parzyste ($\lfloor n/2 \rfloor$); $5 \to 6$: +52% w b)
brakujący filozof	jak wyżej	jak wyżej	$5 \to 4$: bez zmian ($\lfloor n/2 \rfloor = 2$)
lokaj z limitem $\geq n$	możliwa (rozwiązanie trywialne)	–	–
lokaj z limitem $< n - 1$	nie	więcej czekających u lokaja	mniejsza (do -24%)
zapasowy widelec na środku	nie, nawet dla a)	możliwe, wszyscy rywalizują o zapasowy	do 3 jedzących zamiast 2, ale wspólny monitor
widelec w pierścieniu	nie, nawet dla a) (łańcuch)	tak, filozofowie na pozycjach parzystych faworyzowani	najwyższa przy dużej rywalizacji, kosztem sprawiedliwości
brakujący widelec, awaria filozofa	–	pewne dla sąsiadów, w a), c), asym. rozchodzi się łańcuchem	spada
filozof ignorujący protokół	możliwa w a), c), asym.	–	–

Tabela 2: Skutki zmian konfiguracji problemu

7 Wnioski

Rozwiązanie trywialne prowadzi do blokady. Przy dużej rywalizacji następuje ona w ułamku milisekundy, a przy długim myśleniu może nie wystąpić przez długi czas, co czyni błąd trudnym do wykrycia w testach.

Rozwiązanie z widelcami podnoszonymi jednocześnie nie może się zablokować, ale może zagłodzić filozofa, którego sąsiedzi jedzą na zmianę. Gdy jedzenie trwa dłużej niż synchronizacja, jest najwydajniejsze ze wszystkich, bo nikt nie trzyma widelca bez jedzenia. Jego słabością jest jeden monitor stołu i `notifyAll()`, co widać, gdy jedzenie jest krótkie, a filozofów dużo.

Lokaj usuwa blokadę najmniejszą zmianą rozwiązania trywialnego, ale jest najwolniejszy: wspólny monitor jest wąskim gardłem, a przy dużej rywalizacji ograniczenie do $n - 1$ filozofów nie zapobiega długim łańcuchom oczekiwania na widelce.

Rozwiązanie asymetryczne nie ma wspólnego punktu synchronizacji i ma najmniejszy koszt synchronizacji, ale przy dużej rywalizacji jest niesprawiedliwe.

Na wydajność wpływają więc przede wszystkim: stopień rywalizacji (stosunek czasu myślenia do jedzenia), to, czy filozof może trzymać widelec bez jedzenia, obecność wspólnego monitora (stół w b), lokaj w c)) oraz liczba niepotrzebnie budzonych wątków. Gwarancji braku zagłodzenia nie daje żadne z rozwiązań opartych na monitorach Javy, bo nie zachowują one kolejności oczekujących. Uzyskanie jej wymagałoby kolejki FIFO lub priorytetu dla najdłużej głodnego filozofa, kosztem przepustowości, podobnie jak kolejka biletowa w laboratorium 4.

Zmiany konfiguracji (rozdział 6) pokazują, że poprawność rozwiązań zależy od ich założeń. Lokaj jest poprawny tylko wtedy, gdy jego limit jest mniejszy od liczby filozofów, a rozwiązanie asymetryczne – gdy

wszyscy przestrzegają tego samego porządku widelców. Dodatkowy widelec może usunąć blokadę nawet z rozwiązania trywialnego i zwiększyć liczbę jedzących naraz, ale zależnie od położenia faworyzuje część filozofów. Dodanie filozofa nie usuwa blokady, a zmienia liczbę filozofów, którzy mogą jeść jednocześnie ($\lfloor n/2 \rfloor$), oraz możliwość zagłodzenia co drugiego filozofa przy parzystym n.

Literatura

[1] E.W. Dijkstra: *Hierarchical ordering of sequential processes.* Acta Informatica 1(2), 1971.

[2] E.G. Coffman, M. Elphick, A. Shoshani: *System Deadlocks.* ACM Computing Surveys 3(2), 1971.

[3] `https://home.agh.edu.pl/~funika/tw/lab5/`

[4] `https://docs.oracle.com/javase/specs/jls/se27/html/jls-17.html`